Chapter 4 · Lesson 2 — The Orchestrator Pattern

Chapter 4 · Lesson 2 — The Orchestrator Pattern

> *Goal: build the real `<SnookerFantasyLeague>` orchestrator — hero header, sticky tab bar, five tab buttons, conditional tab rendering, and the state that ties them together. By the end of this lesson, the app feels like an app.*

What is an "orchestrator" component?

In real React apps you'll see a pattern called **the orchestrator** (also "container component", "smart component", "shell component" — same idea, different vocabulary).

The orchestrator's job:

1. **Own** the top-level state (`activeTab`, `selectedTeam`).
2. **Compute** the derived data (`teamsWithScores`).
3. **Decide** which child to render based on the state.
4. **Pass** state down as props and setters down as callbacks.

It typically does **not** render very much JSX itself — it just glues other components together.

This pattern shows up in literally every React codebase past five components. It's worth getting right early.

Step 1: rewrite `SnookerFantasyLeague.tsx` as an orchestrator

Replace the stub from Chapter 3 with this:

```
'use client';

import { useState, useMemo } from 'react';
```

```
import { Trophy, Calendar, Target, Users, BarChart3 } from 'lucide-react';
import { TEAMS } from '@/lib/teams';
import { calculateTeamScores } from '@/lib/scoring';
import type { TeamWithScores } from '@/lib/types';
import { BALL_COLORS } from '@/lib/constants';
import StandingsTab from './tabs/StandingsTab';
import MatchesTab from './tabs/MatchesTab';
import PredictionsTab from './tabs/PredictionsTab';
import PlayersTab from './tabs/PlayersTab';
import AnalyticsTab from './tabs/AnalyticsTab';

type TabId = 'standings' | 'matches' | 'predictions' | 'players' | 'analytic

export default function SnookerFantasyLeague() {
  const [activeTab, setActiveTab] = useState<TabId>('standings');
  const [selectedTeam, setSelectedTeam] = useState<TeamWithScores | null>(nu

  const teamsWithScores = useMemo<TeamWithScores[]>(() => {
    return TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))
      .sort((a, b) => b.scores.total - a.scores.total);
  }, []);

  const tabs: { id: TabId; label: string; icon: typeof Trophy }[] = [
    { id: 'standings',   label: 'Standings',   icon: Trophy },
    { id: 'matches',     label: 'Matches',     icon: Calendar },
    { id: 'predictions', label: 'Predictions', icon: Target },
    { id: 'players',     label: 'Players',     icon: Users },
    { id: 'analytics',   label: 'Analytics',   icon: BarChart3 },
  ];

  return (
    <div style={{ /* ...big background gradient... */ }}>
      {/* Hero header */}
      <Hero />
```

```
      {/* Sticky tab bar */}
      <nav style={{ /* ... */ }}>
        {tabs.map(tab => {
          const Icon = tab.icon;
          const active = activeTab === tab.id;
          return (
            <button
              key={tab.id}
              onClick={() => setActiveTab(tab.id)}
              style={{
                /* active vs inactive styling */
                background: active ? '#FFF8E7' : 'transparent',
                color: active ? '#0F5132' : '#6B7280',
                borderBottom: active ? '4px solid #DC2626' : '4px solid trans
                /* ... */
              }}
            >
              <Icon size={20} />
              {tab.label}
            </button>
          );
        })}
      </nav>

      {/* Active tab content */}
      <main style={{ maxWidth: 1400, margin: '0 auto', padding: '32px 24px'
        {activeTab === 'standings' && (
          <StandingsTab
            teams={teamsWithScores}
            onTeamClick={(t) => {
              setSelectedTeam(t);
              setActiveTab('predictions');
            }}
          />
        )}
```

```
        {activeTab === 'matches' && <MatchesTab />}
        {activeTab === 'predictions' && (
          <PredictionsTab
            teams={teamsWithScores}
            selectedTeam={selectedTeam}
            setSelectedTeam={setSelectedTeam}
          />
        )}
        {activeTab === 'players' && <PlayersTab teams={teamsWithScores} />}
        {activeTab === 'analytics' && <AnalyticsTab teams={teamsWithScores}
      </main>
    </div>
  );
}
```

I've abbreviated the Hero and outer styling (full version is in `components/SnookerFantasyLeague.t`
The skeleton is what matters. Walk through every part below.

Walking the orchestrator, top to bottom

Line 1: `'use client'`

Required because we use `useState`. Without it, Next.js would try to render this as a server component and crash on the first `useState` call.

Lines 3–13: imports

Standard imports. Five tab components, five icons, the data and helpers. **Order matters slightly**: third-party (`react`, `lucide-react`), then internal (`@/lib/...`), then local (`./tabs/...`). Keeping that order makes the imports readable at a glance.

Lines 15: a string-union type for tab IDs

```
type TabId = 'standings' | 'matches' | 'predictions' | 'players' | 'analytic
```

Five strings, no others. TypeScript will reject any other value.

This is **the cheapest, most underrated TypeScript pattern**: union types of string literals. They give you autocomplete and typo-protection for free. Compare to `string` everywhere — you'd have no protection against `setActiveTab('predicions')` (note the typo).

Use string unions for any value that has a fixed, small set of options. Round IDs (`'r1' | 'r2' | 'qf' | 'sf' | 'f'`), tab IDs, status enums (`'idle' | 'loading' | 'success' | 'error'`), etc.

Lines 18–19: the two state cells

```
const [activeTab, setActiveTab] = useState<TabId>('standings');
const [selectedTeam, setSelectedTeam] = useState<TeamWithScores | null>(null
```

Two `useState` calls. Each gets:

- An explicit type parameter (`<TabId>`, `<TeamWithScores | null>`) — useful when the initial value alone doesn't pin down the type.
  - `useState<TabId>('standings')` — the initial value `'standings'` could be inferred as `'standings'` (a literal type), too narrow. Adding `<TabId>` widens it to "any tab ID".
  - `useState<TeamWithScores | null>(null)` — without the type param, TS would infer `null` as the type forever, and you'd get a "can't assign TeamWithScores to null" error when you tried to set it.
- An initial value (`'standings'`, `null`).

Two pieces of state. That's the whole app. Compare to the imagined Redux version with reducers, actions, slices, dispatchers, selectors. Two `useStates`, no library, no boilerplate.

Lines 21–24: the memoized derived data

```
const teamsWithScores = useMemo<TeamWithScores[]>(() => {
  return TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))
    .sort((a, b) => b.scores.total - a.scores.total);
}, []);
```

`useMemo` is Lesson 3 of this chapter. For now, the short version: it caches a computed value across renders, only recomputing when the dependency array changes. The empty

array `[]` means "never recompute" — `teamsWithScores` is computed once on mount and reused on every subsequent render.

Why memoize? Two reasons:

1. **Performance** — without `useMemo`, every tab click would re-run `calculateTeamScores` for all 8 teams. Wasteful.
2. **Reference stability** — without `useMemo`, the `teamsWithScores` array would be a *new array reference* on every render. That can cause downstream `useMemo` and `useEffect` re-runs in child components. Stable references are quietly important.

We'll see both reasons play out in Lesson 3.

Lines 26–32: the tabs configuration

```
const tabs: { id: TabId; label: string; icon: typeof Trophy }[] = [
  { id: 'standings',   label: 'Standings',   icon: Trophy },
  { id: 'matches',     label: 'Matches',     icon: Calendar },
  // ...
];
```

A **data-driven nav bar**. Define an array of tab descriptors; map over it to render buttons. Adding a new tab is one line in this array (plus the corresponding tab component, of course). Removing one is also a one-line edit.

This is dramatically better than hard-coding five `<button>` elements with their labels. **Configuration over repetition** is a core principle.

The `icon: typeof Trophy` type — that's TypeScript syntax for *"the same type as the value `Trophy`"*. Each Lucide icon has the same component type, so we just use one as the canonical example.

Lines 35–80: the JSX

The orchestrator returns three top-level pieces:

1. **The Hero** — a big gradient banner with the app name and the champion badge. (Pure decoration; could be its own component.)
2. **The nav** — sticky tab bar.
3. **The main** — switches on `activeTab` to render the right tab.

Walk the tab bar:

```
{tabs.map(tab => {
  const Icon = tab.icon;
  const active = activeTab === tab.id;
  return (
    <button
      key={tab.id}
      onClick={() => setActiveTab(tab.id)}
      style={{
        background: active ? '#FFF8E7' : 'transparent',
        color: active ? '#0F5132' : '#6B7280',
        borderBottom: active ? '4px solid #DC2626' : '4px solid transparent'
      }}
    >
      <Icon size={20} />
      {tab.label}
    </button>
  );
})}
```

`tab.icon` is renamed to local `Icon` (capitalized so JSX recognizes it). The `active` boolean controls every visual difference: background, color, border. Three branches in one component, all tied to one state value.

Click a tab → setter fires → state changes → orchestrator re-renders → all five buttons re-render with new `active` values → DOM updates. Six steps, one click. Memorize the loop.

The cross-tab interaction

This is the part that makes the app feel like an app:

```
{activeTab === 'standings' && (
  <StandingsTab
    teams={teamsWithScores}
    onTeamClick={(t) => {
      setSelectedTeam(t);
      setActiveTab('predictions');
```

```
    }}
  />
)}
```

When the user clicks a team row in `<StandingsTab>`, the callback runs **two state setters back-to-back**:

1. `setSelectedTeam(t)` — store the clicked team.
2. `setActiveTab('predictions')` — switch the visible tab.

React's automatic batching collapses these into **one re-render**. Both states change atomically. The user clicks → both states update → orchestrator re-renders → predictions tab is shown with the right team selected.

That's the orchestrator pattern in action. Two setter calls in one click, multiple components affected, one consistent re-render. State at the top, behavior glued together at the top, leaves are pure.

The && short-circuit

```
{activeTab === 'standings' && <StandingsTab ... />}
```

This is JavaScript's logical && operator. If the left side is **truthy**, the right side is evaluated and returned. If the left side is **falsy**, the right side is short-circuited and && returns the left side (the falsy value).

For our use case: if `activeTab === 'standings'` is `true`, we get the `<StandingsTab />` element. If `false`, we get `false` itself, which React renders as nothing. **That's how conditional rendering works in JSX**: render or don't render based on a condition.

The classic && gotcha is when the left side is `0`:

```
{count && <Badge>{count}</Badge>}
```

If `count` is `0`, React renders… `0`. Because `0 && anything` is `0`, and React renders `0` as text! Use `{count > 0 && ...}` or a ternary `{count ? <Badge>{count}</Badge> : null}` instead.

Updating the tab components to receive props

`<StandingsTab>` from Chapter 3 used `TEAMS` directly. Now it needs to receive teams from the parent:

```
import type { TeamWithScores } from '@/lib/types';
import LeagueTable from '../standings/LeagueTable';

interface StandingsTabProps {
  teams: TeamWithScores[];
  onTeamClick?: (team: TeamWithScores) => void;
}

export default function StandingsTab({ teams, onTeamClick }: StandingsTabPro
  return (
    <div>
      {/* Stat cards above */}
      <LeagueTable teams={teams} onTeamClick={onTeamClick} />
    </div>
  );
}
```

What changed: `<StandingsTab>` no longer imports `TEAMS` or calls `calculateTeamScores`. It receives `teams` from the orchestrator. **The orchestrator is now the single source of truth.**

Stub the other four tabs

For the orchestrator to compile, all five tab files must exist. Create stubs for the four we haven't built:

```
// components/tabs/MatchesTab.tsx
export default function MatchesTab() {
  return (
    <div style={{ padding: 32, textAlign: 'center', color: '#6B7280' }}>
      <h2>Matches</h2>
      <p>Coming soon — Chapter 5.</p>
    </div>
```

```
  );
}
```

Repeat for `PredictionsTab.tsx`, `PlayersTab.tsx`, `AnalyticsTab.tsx`. Each takes whatever props the orchestrator passes (or none) and renders a placeholder. The compiler is happy; the runtime is happy; you can switch tabs and see "Coming soon" placeholders for the unimplemented ones. Save and refresh.

You should now be able to click between tabs. Standings shows the full table. The others show placeholders. The hero header is sticky, the tab bar is sticky, the main content scrolls.

Why does state live where it lives?

Three pieces of state in this app:

1. **`activeTab`** — used by the orchestrator (to decide which tab to render) and the tab buttons (to highlight the active one). Lowest common ancestor: the orchestrator. **Lives there.**
2. **`selectedTeam`** — set by `<StandingsTab>` (via `onTeamClick`) and read by `<PredictionsTab>`. Lowest common ancestor: the orchestrator. **Lives there.**
3. **`round`** (inside `<MatchesTab>`) — set by tab strip clicks inside MatchesTab, read by MatchesTab itself. Lowest common ancestor: MatchesTab. **Lives in MatchesTab.**

The pattern: **each piece of state lives at the lowest component that needs to read or write it.** Higher than that = unnecessary prop drilling. Lower than that = inability to share.

This is "**lifting state up**" — when two siblings need to share, you lift the state to their parent. **Lifting state is the canonical solution for sibling communication in React.** Don't pass functions through siblings, don't use refs, don't reach for a global store. Just lift.

Why this is interview gold

If an interviewer hands you a multi-screen React problem on a whiteboard and you start by asking *"what state do we have, and where does it live?"* — you've already aced the first 60% of the interview. **The vast majority of senior React interviews are about state location**, not about hooks or lifecycle methods or rendering tricks.

Concretely, in interviews:

- *"How would you let one component trigger a change in another?"* → "Lift the state to their common ancestor; pass the value down as a prop and a setter down as a callback."
- *"How would you avoid prop drilling for deeply nested data?"* → "Three levels is fine; for more, consider Context or a state library. But I'd default to drilling first because it's explicit."
- *"How would you share state between two routes?"* → "Move it above the router. URL params if it should be bookmarkable. Top-level state if not."

All of those answers reduce to *"state lives at the lowest common ancestor of its readers and writers."* Memorize that one rule and you've got most of state management licked.

Why we DON'T use Context yet

Context is React's built-in alternative to prop drilling. It looks like this:

```
const TeamsContext = createContext<TeamWithScores[]>([]);

// Provider in the orchestrator
<TeamsContext.Provider value={teamsWithScores}>
  {children}
</TeamsContext.Provider>

// Consumer anywhere below
const teams = useContext(TeamsContext);
```

We don't use it. Why?

- We have **3 levels** of nesting at most (orchestrator → tab → leaf). Three levels of `teams` props is fine.
- Context has its own pitfalls: every Provider value change re-renders **all consumers** unless you split it carefully. Easy to misuse.
- Context **hides** the data flow. With prop drilling you can `Cmd+F "teams"` and see exactly where it goes. With Context, you can't.

Context is for cross-cutting concerns — theme, auth user, locale — not for general state. The internet's love affair with Context is misplaced for many cases.

If we ever needed deeply-nested access, **Zustand** would be a better choice for our app

size. But we don't, so we don't.

> **Senior interview answer:** *"Context is great for theme/auth/locale. For application data, I default to prop drilling because it's explicit and Cmd+F-able. If drilling becomes painful, I'd reach for Zustand or Jotai before Context."*

Vibe prompt you would have used

> *"Build the `<SnookerFantasyLeague>` orchestrator at `components/SnookerFantasyLeague.tsx`. Mark with `'use client'`. Two `useState`s: `activeTab: 'standings' | 'matches' | 'predictions' | 'players' | 'analytics'` (default `'standings'`), `selectedTeam: TeamWithScores | null` (default `null`). One `useMemo` that maps `TEAMS` through `calculateTeamScores` and sorts by `.scores.total` desc. Render: a hero header with the app title and champion badge, a sticky tab nav bar with five Lucide-icon buttons, then a `<main>` that conditionally renders the matching tab component. From `<StandingsTab>`'s `onTeamClick`, set `selectedTeam` AND switch `activeTab` to `'predictions'`. Stub `<MatchesTab>`, `<PredictionsTab>`, `<PlayersTab>`, `<AnalyticsTab>` as one-line 'Coming soon' placeholders.* ***Don't use Context. Don't use Redux.*** *Just useState + props."*

The "**don't use X**" instructions are critical here. The default LLM reflex is to introduce Context or a state library "for cleanliness" — and you'll get back 200 lines of boilerplate. Forbid it.

CHECK YOURSELF

1. **`activeTab` is state at the orchestrator level. `round` (inside MatchesTab) is state at the tab level. What's the principle that determines which level each lives at?**
2. **Walk through every render that happens when the user clicks a team row in `<StandingsTab>`.** How many setter calls? How many re-renders? Which components re-render?
3. **Why does the orchestrator have `'use client'` but `app/layout.tsx` doesn't?** What's the boundary?
4. **You add `console.log('orchestrator')` at the top of `<SnookerFantasyLeague>`'s function body. Then you click "Predictions". How many times does it log?**

5. **A teammate suggests moving `selectedTeam` into `<PredictionsTab>` "to keep state close to where it's used."** Why is this the wrong move? What would break?

When you've answered these, head to **03-useMemo-and-derived-data.md** — the final lesson of this chapter.